

VISÃO DO PRODUTO E DO PROJETO

Versão 1.0

Tabela 1 - Integrantes do Grupo:

Matrícula	Nome	Função (responsabilidade)	Pontos de participação na elaboração
251013624	Mateus Alves Araujo	Gestão de projetos e Documentação	8,33...%
211043772	Vinícius dos Santos Evangelista Dias	Back-end	8,33...%
251012313	Luís Gustavo Ferreira Nunes	Back-end	8,33...%
211062428	Rafael Souto Lopes Laube	Back-end	8,33...%
251040087	Gabriel José Neves de Resende	Analista de Qualidade	8,33...%
251013787	Murilo Paulino	Back-end	8,33...%
251013651	Matheus Estevam Ferreira	Back-end	8,33...%
251035659	Rafael Lima Sant'Ana	Gestão de projetos e Documentação	8,33...%
232028428	Luiz Henrique Pessato da Mota	Front-end	8,33...%
232014306	José Gabriel Maques de Araujo	Front-end	8,33...%

251027835	Rafael Nicolas Freitas Araújo	Análise de qualidade	8,33...%
251037831	Osmar Júnio Mendes Carmo dos Santos	Front-end	8,33...%

Histórico de Revisões

Data	Versão	Descrição	Autor
06/05/2026	1.0	Fim da Primeira Versão do projeto	Todos

Sumário

1	VISÃO GERAL DO PRODUTO	4
1.1	Problema.....	4
1.2	Declaração de Posição do Produto	4
1.3	Objetivos do Produto.....	5
1.4	Tecnologias a Serem Utilizadas	6
2	VISÃO GERAL DO PROJETO	6
2.1	Ciclo de vida do projeto de desenvolvimento de software.....	7
2.2	Organização do Projeto.....	7
2.3	Descrição da Organização.....	8
2.4	Planejamento das Fases e/ou Iterações do Projeto	8
2.5	Descrição do Planejamento	9
2.6	Matriz de Comunicação.....	9
2.7	Gerenciamento de Riscos.....	10
	Critérios de Replanejamento	10
3	PROCESSO DE DESENVOLVIMENTO DE SOFTWARE	10
3.1	Scrum.....	11
4	DECLARAÇÃO DE ESCOPO DO PROJETO.....	13
4.1	Backlog do produto	13
4.2	Perfis.....	13
4.3	Cenários.....	14
4.4	Tabela de Backlog do produto	15
5	MÉTRICAS E MEDIÇÕES.....	19
5.1	GQM de medições.....	19
5.1.1	Questões de medição	19
5.1.2	Métricas, limites e ações	20
5.2	Considerações de verificação e validação	21
6	TESTES DE SOFTWARE.....	21
6.1	Estratégia de testes contendo:	22
6.2	Roteiro de teste:	22
7	REFERÊNCIAS BIBLIOGRÁFICAS	23

VISÃO DO PRODUTO E PROJETO

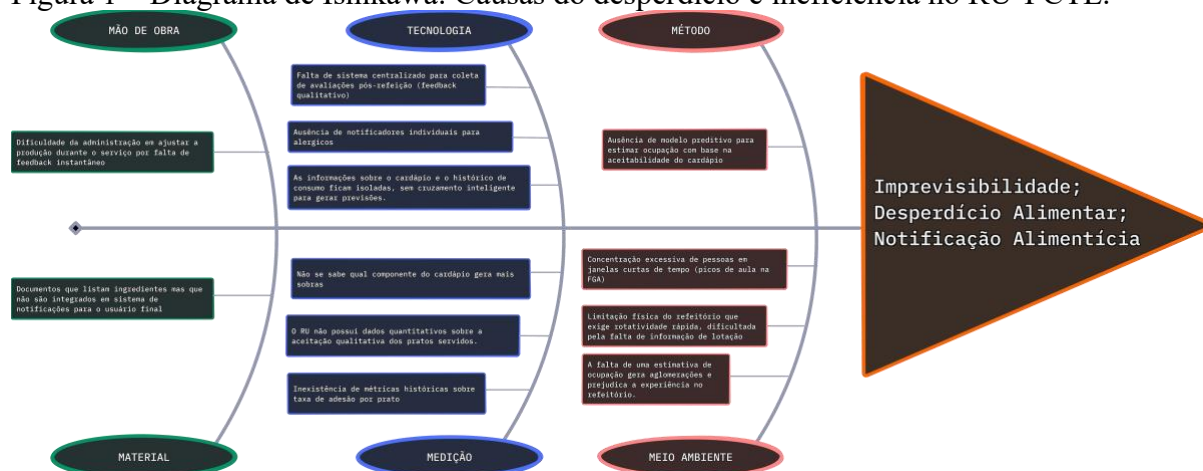
1 VISÃO GERAL DO PRODUTO

1.1 Problema

O Restaurante Universitário (RU) da Faculdade do Gama (FCTE/UnB) é um serviço essencial para a rotina acadêmica. Atualmente, o RU dispõe de um site institucional onde o cardápio é publicado. Porém, esse sistema funciona de forma passiva: a informação flui da administração para o aluno, mas não há um retorno de dados que possa permitir uma gestão dinâmica. Assim, faz-se preciso uma infraestrutura digital onde não apenas informe o que está sendo servido, mas que possa prever a adesão dos usuários e/ou que alerte usuários sobre riscos nutricionais de forma individual e personalizável.

A análise do diagrama de Ishikawa (Figura 1) revela que o problema é multifacetado. No pilar da Tecnologia, destaca-se a falta de um canal para coleta de feedback qualitativo, enquanto em Medição, a inexistência de métricas históricas de adesão impede qualquer planejamento preciso. O Método atual, baseado em janelas curtas de tempo e alta rotatividade, agrava a lotação, uma vez que o aluno não possuirá informações prévias para decidir o melhor horário para sua refeição. Ademais, a categoria Material aponta uma ausência de sistema de notificações que possa auxiliar individualmente na conexão entre as fichas técnicas de ingredientes e o usuário final, visando à proteção de alunos alérgicos.

Figura 1 – Diagrama de Ishikawa: Causas do desperdício e ineficiência no RU-FCTE.



Diante deste cenário, a solução proposta consiste em um sistema inteligente de gestão que utiliza taxas de adesão e preferência para prever comportamento dos usuários. Diferente de sistemas baseados em hardware de monitoramento físico, esta solução foca na análise de dados e no engajamento da comunidade acadêmica da FCTE. Espera-se que o software contribua para a redução direta do desperdício alimentar, ao fornecer à administração estimativas de demanda baseadas no histórico de aceitabilidade de cada prato. Para o estudante, o sistema oferecerá maior conveniência e segurança alimentícia, permitindo o planejamento do horário de refeição conforme lotação estimada e garantindo alertas personalizados sobre riscos nutricionais, transformando a rotina do RU em um processo mais orientado a dados.

1.2 Declaração de Posição do Produto

A equipe realizará um site na Web que visa a avaliação de pratos no RU, a fim de melhorar a gestão de alimentos de seus funcionários. Para os alunos o sistema fornecerá o cardápio

semanal do RU, possibilidade de notificar alimentos que o aluno prefere ou são nocivos para ele e pedirá que o aluno avalie o cardápio do dia. Para os funcionários o sistema fornecerá a média das avaliações do prato e quantos alunos tem alguma das opções como alimento favorito ou nocivo.

Até o momento de realização desse documento não há nenhuma ideia que busca avaliar a qualidade das refeições do Restaurante Universitário, tão pouco integrar alunos com funcionários do RU, o objetivo de criar um sistema que faça isso é:

- Por meio da avaliação da qualidade dos alimentos, os alunos não precisarão gastar tempo na fila para comer algo que muitos não gostaram, evitando filas enormes e a necessidade de funcionários do RU de circularem frequentemente entre a cozinha e o balcão de distribuição.
- Alunos já serão notificados quando estiver sendo servido pratos que o aluno não é recomendado a comer ou que o aluno gosta, por meio desses dados o Restaurante da FCTE terá uma estimativa de quais serão as opções com mais ou menos demanda.

Tabela 2 – O que a aplicação representa para os Alunos?

Para:	Alunos da FCTE
Necessidade:	Saber a qualidade da comida do RU e quando serão servidos alimentos específicos
O AvaliaRU:	É um programa Web
Que:	Oferece formas de os alunos avaliarem os pratos servidos no RU e notifica sobre alimentos favoritos e ingredientes nocivos
Ao contrário:	Evitar desperdiçar tempo em filas
Nosso produto:	Ajude o aluno a decidir se deve realizar refeições no RU

Tabela 3 – O que a aplicação representa para o RU

Para:	Funcionários do RU da FCTE
Necessidade:	Uma forma de economizar os ingredientes disponibilizados
O AvaliaRU:	É um programa Web
Que:	Oferece estimativas da demanda que cada refeição tem
Ao contrário:	Jogar fora ingredientes já cozinhados e que não foram consumidos
Nosso produto:	Permite que o Restaurante não gaste quantidades desnecessárias de alimento

1.3 Objetivos do Produto

O objetivo principal deste projeto é o de desenvolver um sistema inteligente de gestão para o RU da FCTE, utilizando do engajamento dos usuários para transformar a publicação passiva do cardápio em um modelo preditivo de demanda, assim visando reduzir o desperdício

alimentar e aumentar a personalização nutricional dos estudantes, para que os mesmos possam ser orientados de possíveis componentes alérgicos.

Para alcançar o objetivo geral, será preciso realizar os seguintes objetivos secundários:

- **Implementação de um sistema avaliativo qualitativo** o qual possa permitir que os usuários avaliem as refeições para gerar uma taxa de preferência por prato.
- **Desenvolvimento de um algoritmo de estimativa de adesão:** Cruzamento de dados de preferência com o cardápio do dia para projetar o volume de usuários e a lotação do refeitório.
- **Criação de módulo de notificações personalizadas** para alertar automaticamente usuários cadastrados sobre a presença de alérgenos específicos em pratos do dia.
- **Melhorar a transparência informativa** oferecendo uma interface interativa onde o aluno possa consultar o cardápio e a previsão de lotação baseada em alta adesão.

1.4 Tecnologias a Serem Utilizadas

Banco de Dados:

- **SQLite** – Banco de dados leve e mais fácil de usar. Perfeito para iniciantes e para a disciplina de MDS que não se aprofunda em banco de dados.

Linguagens e Frameworks:

- **Typescript** – Superconjunto do JavaScript que adiciona uma tipagem forte à linguagem, a fim de detectar erros durante o tempo de desenvolvimento do código.
- **Next.js** - Framework do Typescript, baseado em React para desenvolvimento full-stack (Back-end e Front-end)
- **HTML** – Principal linguagem de marcação para desenvolvimento web.
- **CSS** – Uma folha de estilo responsável pela estética de uma aplicação web.

Infraestrutura:

- **AWS** - Serviço de hospedagem gratuito da Microsoft que será usado para realizar o deploy da aplicação web.
- **Github** – Plataforma de hospedagem de repositórios Git. Por meio dele os membros da equipe irão compartilhar seus trabalhos.
- **IDEs**- Ambientes de Desenvolvimento Integrado que auxiliam a equipe a desenvolver código.

2 VISÃO GERAL DO PROJETO

2.1 Ciclo de vida do projeto de desenvolvimento de software

O ciclo de vida adotado para o desenvolvimento deste projeto será baseado em metodologias ágeis, utilizando Scrum como estrutura principal de gerenciamento e práticas de Extreme Programming (XP) como suporte ao desenvolvimento técnico.

O Scrum organiza o desenvolvimento em ciclos iterativos e incrementais chamados Sprints, que terão duração de uma semana. A cada Sprint, será entregue uma versão funcional do sistema, permitindo evolução contínua do produto e feedback frequente dos usuários.

O projeto consiste no desenvolvimento de um sistema web utilizando Next.js, com o objetivo de disponibilizar o cardápio semanal do Restaurante Universitário (RU), além de oferecer funcionalidades como:

1. Notificação de usuários sobre alimentos que possam causar alergias ou que não sejam de sua preferência;
2. Cadastro de restrições alimentares e preferências;
3. Avaliação das refeições pelos usuários, permitindo feedback aos funcionários do RU.

Durante o ciclo de vida do projeto, as seguintes etapas do Scrum serão aplicadas:

1. Planejamento da Sprint (Sprint Planning): definição das funcionalidades a serem desenvolvidas com base no backlog do produto, priorizando itens como exibição do cardápio, sistema de notificações e avaliações;
2. Execução da Sprint: desenvolvimento das funcionalidades utilizando Next.js e persistência de dados;
3. Revisão da Sprint (Sprint Review): validação das funcionalidades implementadas, como exibição do cardápio semanal e sistema de alertas personalizados;
4. Retrospectiva da Sprint: análise do processo de desenvolvimento e identificação de melhorias.

Além disso, serão adotadas práticas de XP para garantir maior qualidade no desenvolvimento, tais como:

1. Desenvolvimento orientado a testes (quando aplicável);
2. Refatoração contínua do código;
3. Integração contínua;
4. Padronização de código;
5. Programação colaborativa entre os membros da equipe.

A escolha desse ciclo de vida se justifica pela natureza dinâmica do projeto, que envolve requisitos que podem evoluir ao longo do desenvolvimento, especialmente relacionados às preferências dos usuários e às funcionalidades de notificação e avaliação. O uso de metodologias ágeis permite maior flexibilidade, entregas incrementais e adaptação às necessidades dos usuários.

Além disso, a interação contínua com possíveis usuários (estudantes e funcionários do RU) possibilita validar rapidamente as funcionalidades implementadas, garantindo que o sistema atenda às necessidades reais do contexto acadêmico.

2.2 Organização do Projeto

Papel	Atribuições	Participantes
Back-End	Desenvolver a interface do sistema utilizando Next.js, implementar telas como cardápio semanal, cadastro de usuários e sistema de avaliações	Murilo Paulino, Rafael Laube, Vinícius dos Santos, Luis Gustavo
Front-End	Implementar regras de negócio, APIs, sistema de notificações e integração com banco de dados PostgreSQL	Osmar Júnio,
Analista de Qualidade	Garantir a qualidade do sistema, elaborar e executar testes, validar funcionalidades e verificar conformidade com os requisitos	Gabriel José, Rafael Nicolas
Gestão de Projetos e Documentação	Organizar as Sprints, acompanhar o progresso da equipe, remover impedimentos e garantir aplicação das metodologias ágeis	Mateus Araújo, Rafael Lima
Cliente	Validar entregas e fornecer feedback sobre o sistema	Usuários do sistema

2.3 Descrição da Organização

A equipe adota uma abordagem colaborativa, onde, apesar da divisão de papéis, todos os membros podem contribuir em diferentes áreas quando necessário. Essa flexibilidade é importante para garantir maior produtividade e aprendizado coletivo.

O papel de frontend é responsável pela experiência do usuário, garantindo que o sistema seja intuitivo e funcional. O backend cuida da lógica do sistema, incluindo o processamento de dados e envio de notificações personalizadas com base nas preferências alimentares dos usuários.

O analista de qualidade atua durante todo o processo, garantindo que as funcionalidades desenvolvidas estejam corretas e atendam aos requisitos definidos. Já a gestão de projetos assegura o bom andamento das Sprints e a aplicação das práticas ágeis.

Por fim, o cliente desempenha um papel fundamental na validação do sistema, garantindo que o produto final atenda às necessidades dos usuários do Restaurante Universitário.

2.4 Planejamento das Fases e/ou Iterações do Projeto

Sprint	Produto (Entrega)	Data Início	Data Fim	Entregas	Responsaveis	% conclusão
Sprint 1	Estudo e definição do projeto	03/05/2026	09/05/2026	Levantamento de requisitos, estudo de Next.js, definição da arquitetura inicial	Equipe	0%
Sprint 2	Protótipo e MVP inicial	10/05/2026	16/05/2026	Protótipo das telas, estrutura básica do sistema,	Equipe	0%

				cadastro de usuários		
Sprint 3	Cardápio semanal	17/05/2026	23/05/2026	Funcionalidade de exibição do cardápio e do Cadastro de pratos, pelo gestor do RU integradas ao banco de dados	Equipe	0%
Sprint 4	Preferências e restrições	24/05/2026	30/05/2026	Cadastro de alergias e preferências alimentares	Equipe	0%
Sprint 5	Sistema de notificações	31/05/2026	06/06/2026	Alertas personalizados para usuários	Equipe	0%
Sprint 6	Avaliação das refeições	07/06/2026	13/06/2026	Sistema de feedback das refeições	Equipe	0%
Sprint 7	Testes e ajustes finais	14/06/2026	20/06/2026	Correções, testes gerais e melhorias	Equipe	0%

2.5 Descrição do Planejamento

As Sprints iniciais são focadas na adaptação da equipe às tecnologias utilizadas no projeto, especialmente Next.js, garantindo que todos os membros tenham conhecimento suficiente para contribuir de forma efetiva.

A partir da segunda Sprint, inicia-se o desenvolvimento do MVP (Produto Mínimo Viável), com funcionalidades essenciais como visualização do cardápio e cadastro de usuários.

As funcionalidades mais complexas, como notificações personalizadas e sistema de avaliação, são desenvolvidas nas Sprints seguintes, após a consolidação da base do sistema.

Por fim, a última Sprint é dedicada à realização de testes, correções e melhorias, assegurando a qualidade do produto antes da entrega final.

2.6 Matriz de Comunicação

Descrição	Área / Envolvidos	Periodicidade	Produtos Gerados
Acompanhamento das atividades em andamento (tarefas do sistema RU)	Equipe do Projeto	Semanal	Ata de reunião
Acompanhamento de riscos, compromissos, ações pendentes e indicadores	Equipe do Projeto	Semanal	Relatório de situação do projeto

Comunicação e atualização geral do projeto	Equipe do Projeto	Quinzenal	Relatório de status do projeto
Acompanhamento e validação do progresso	Equipe do Projeto	Semanal	Ata de reunião + Relatório de situação do projeto

2.7 Gerenciamento de Riscos

[O Gerenciamento de Riscos consiste em tarefas, como:

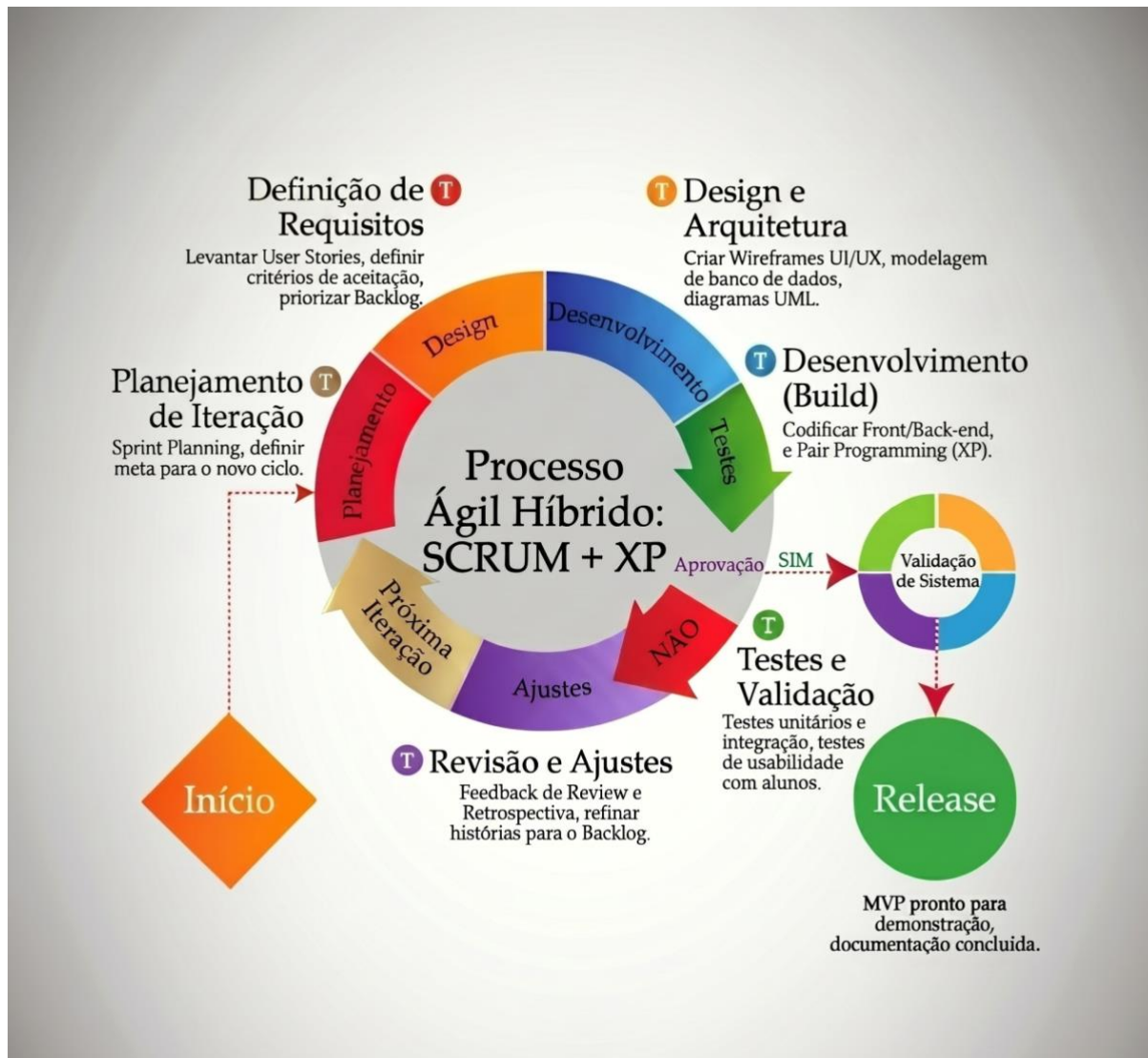
Risco	Grau de Exposição	Mitigação	Plano de Contingência
Avaliações não serem registradas corretamente	Médio	Testes no banco de dados	Correção e reprocessamento
Falha nas notificações de favoritos	Alto	Testes de envio de notificações	Reenvio manual ou aviso no sistema
Erro na identificação de restrições alimentares	Alto	Validação rigorosa dos dados	Bloquear exibição até corrigir
Dados incorretos no banco	Alto	Validação e testes	Restaurar backup
Lentidão ao buscar pratos favoritos	Médio	Otimizar consultas	Simplificar busca

Critérios de Replanejamento

- Falhas nas notificações (principal funcionalidade)
- Erros críticos envolvendo restrições alimentares
- Problemas no registro de avaliações
- Mudanças nos requisitos do RU
- Problemas de desempenho no sistema
- Dificuldades na integração com banco de dados

3 PROCESSO DE DESENVOLVIMENTO DE SOFTWARE

Figura 2 – Processo de Desenvolvimento de Software



Este tópico detalha as metodologias e o ciclo de vida do Software que foram adotados pelo grupo para o desenvolvimento do sistema de melhorias do Restaurante Universitário (RU) da UnB-Gama. A equipe optou por uma abordagem de metodologias ágil híbrida, integrando o Scrum para a gestão do produto, organização das entregas e adaptabilidade, e o Extreme Programming (XP) para assegurar a qualidade técnica e a agilidade na escrita do código. Esta escolha visa garantir que o desenvolvimento esteja alinhado às necessidades dos usuários, mantendo um ritmo sustentável de entregas de valor.

3.1 Scrum

- **Gestão Visual (Kanban):** Utilização de ferramentas como GitHub Projects ou Trello para monitoramento de tarefas (To Do, Doing, Testing, Done).
- **Equipes:** Divisão interna entre Front-End, Back-End e Banco de Dados e Design.
- **Sprints:** Ciclos com duração de uma semana. Cada sprint deverá ter o objetivo de incrementar novas funcionalidades para o código ao seu final.
- **Sprint Planning:** Feito no início da Sprint, responsável por definir o que deve ser realizado ao longo da Sprint.
- **Sprint Review:** Feito ao final de toda Sprint, para avaliar o resultado da Sprint e determinar futuras mudanças, inclusive no Product Backlog.

3.2 Extreme Programming (XP)

- **Feedback:** Estabelecimento de ciclos constantes de comunicação entre os desenvolvedores e os clientes ou stakeholders para avaliações constantes dos incrementos, através de testes e revisões.
- **Pair Programming:** Técnica de desenvolvimento onde dois programadores trabalham juntos em uma única estação de trabalho. Enquanto um é responsável pela codificação, o outro observa o código sendo produzido e tenta pensar em formas de simplificar e melhorar o código. As duplas invertem os papéis constantemente e alternam com outras duplas da equipe, assim todos os membros podem ter conhecimento de todas as partes do sistema
- **Refatoramento:** São constantes melhorias na estrutura interna do código sem alterar seu comportamento externo. O objetivo é simplificar o design, mais bem estruturado, tornar o código mais legível e fácil de manter, garantindo a saúde técnica do projeto a longo prazo.
- **Integração contínua:** O código de uma funcionalidade nova, uma vez pronto deve ser integrado ao repositório principal, cada integração é acompanhada por testes, permitindo que erros sejam percebidos e ajustados, assim garantindo que o software esteja sempre em estado funcional.
- **Padrões de código:** Definição de regras e convenções de estilo do código que devem ser seguidos pelos membros da equipe. Isso garante que todos programem da mesma forma, facilitando leitura e manutenção.

4 DECLARAÇÃO DE ESCOPO DO PROJETO

4.1 Backlog do produto

Os perfis foram divididos em Administradores, Estudantes e Gestor do RU. Essa divisão foi decidida pelo grupo a partir da elicitação dos requisitos e após o envio de um formulário aos estudantes da UnB, e outro formulário aos funcionários do RU, ambos contendo perguntas relacionadas à melhoria do restaurante.

Os estudantes representam os alunos da FCTE que buscam verificar o cardápio do RU, bem como receber informações nutricionais sobre os alimentos produzidos naquele dia (RF03 a RF

Os gestores do RU buscam verificar no sistema a avaliação dos pratos feita pelos estudantes, de modo a promover a melhoria do serviço prestados pelo restaurante aos estudantes e funcionários da FCTE. Os administradores são os responsáveis por gerenciar os usuários, assim como fazer manutenções.

4.2 Perfis

Tabela 4: Perfis de acesso

#	Nome do perfil	Características do perfil	Permissões de acesso
1	Administrador	Membro da equipe responsável pelo gerenciamento e manutenção do sistema	Gerenciar contas de usuários, perfis e realizar backups do banco SQLite
2	Estudante (FCTE)	Usuário principal, que consome as refeições e busca informações acerca da comida servida no RU	Visualizar cardápio, configurar restrições/favoritos e avaliar refeições
3	Gestor RU	Funcionário ou administrador responsável pelo planejamento do cardápio ofertado no RU	Visualizar as avaliações, gerenciar os pratos ofertados

Tabela 5: Elicitação de requisitos

Perfil	Técnica de Elicitação	Justificativa
Estudante (FCTE)	Formulário	Coleta em tempo hábil de dados quantitativos e qualitativos acerca do interesse na aplicação proposta e sugestões de funcionalidades a serem adicionadas
Gestor (RU)	Formulário	Coleta em tempo hábil de dados quantitativos e qualitativos acerca do interesse na aplicação proposta e sugestões de funcionalidades a serem adicionadas

4.3 Cenários

Tabela 6: Cenários funcionais

Sistema: AvaliaRU – Cenários funcionais		
Numeração do cenário	Nome do cenário	Sprints
CN01	Autenticação de perfil	2
CN02	Visualização do cardápio	3
CN03	Gestão de favoritos	4
CN04	Gestão de restrições alimentares	4
CN05	Notificações ao estudante	5
CN06	Avaliação pós-refeição	6
CN07	Gestão de demanda	6
CN08	Gestão de pratos	3

4.4 Tabela de Backlog do produto

Tabela 7: Backlog do produto

Sistema: AvaliaRU – Backlog do produto						
Numeração	Sprint	Nome do requisito	Tipo de requisito	Priorização	Descrição sucinta do requisito	User histories (U.S.) associadas
CN01/RF01	2	Cadastro do Usuário	Funcional	Must	Permite o cadastro do usuário	Eu como estudante da FCTE quero me cadastrar no AvaliaRU para obter o acesso ao sistema.
CN01/RF02	2	Autenticação do Usuário	Funcional	Must	Permite o acesso do usuário (estudantes, gestores do RU e administradores) ao sistema	Eu como estudante ou gestor do RU ou administrador quero me autenticar no sistema para realizar as operações no sistema
CN01/RF03	2	Saída do Usuário	Funcional	Must	Permite encerrar a sessão do usuário (estudantes, gestores do RU e administradores) no sistema	Eu como estudante, gestor do RU ou administrador quero encerrar a minha sessão de usuário para não seja possível utilizar o sistema.
CN08 / RF04	3	Listar pratos gestão	Funcional	Must	Visualizar os pratos	Eu como gestor do RU, ou como administrador quero visualizar a lista de pratos cadastrados para manter a lista correta de pratos fornecidos pelo RU.
CN08 / RF05	3	Cadastrar pratos	Funcional	Must	Cadastro um prato	Eu como gestor do RU ou administrador quero cadastrar um prato para permitir a avaliação dos estudantes e ver a lista de pratos cadastrados

						para garantir que o prato foi adicionado com sucesso.
CN08 / RF06	3	Remover pratos gestão	Funcional	Must	Remover um prato da lista de pratos disponíveis	Eu como gestor do RU ou administrador quero remover um prato da lista para garantir que a ela esteja de acordo com os pratos fornecidos pelo RU
CN02/RF07	3	Mostrar cardápio	Funcional	Must	Exibe o cardápio	Eu como estudante quero visualizar o cardápio do RU para saber se quero almoçar no RU ou não.
CN02/RF08	4	Listar pratos estudante	Funcional	Must	Exibe os pratos e as médias de avaliação dos pratos	Eu como estudante quero ver a lista de pratos ofertados pelo RU e as avaliações médias, para realizar as ações de avaliar, adicionar ou remover na lista de favoritos ou de restrições.
CN03/RF09	4	Adicionar prato favorito	Funcional	Must	Registra o prato na lista de pratos favoritos do estudante	Eu como estudante quero adicionar um prato na minha lista de pratos favoritos para receber notificações caso o prato seja disponibilizado naquele dia. O prato deve ser adicionado à minha lista de favoritos.
CN03/RF10	4	Remover prato favorito	Funcional	Must	Remover o prato da lista de favoritos do estudante	Eu como estudante quero remover um prato da lista de favoritos para não receber mais notificações de prato favorito. O prato deve ser

						removido da lista de pratos favoritos.
CN04/RF10	4	Adicionar prato como restrição alimentar	Funcional	Must	Registrar o prato na lista de restrições alimentares do estudante	Eu como estudante quero adicionar um prato na minha lista de restrições alimentares para receber notificações caso o prato seja disponibilizado pelo RU, com um aviso de restrição alimentar.
CN04/RF11	4	Remover prato como restrição alimentar	Funcional	Must	Remove o prato da lista de restrições do estudante.	Eu como estudante quero remover um prato da minha lista de restrições para não receber mais notificações de restrição alimentar desse prato.
CN05/RF12	5	Notificar alimento favorito	Funcional	Must	Notifica o usuário quando um ou mais dos pratos na lista de favoritos for disponibilizado pelo RU naquele dia.	Eu como estudante quero receber uma notificação avisando sobre meu prato favorito estar disponível no RU naquele dia para decidir se almoço no RU ou não.
CN05/RF13	5	Notificar restrição alimentar	Funcional	Must	Notifica o usuário quando um ou mais dos pratos na lista de restrições for disponibilizado pelo RU naquele dia.	Eu como estudante quer ser avisado caso um prato disponibilizado pelo RU naquele dia esteja na minha lista de restrição para ficar informado sobre as restrições.
CN06/RF14	6	Listar os pratos mais favoritados	Funcional	Must	Exibe quais os pratos mais favoritados pelos usuários	Eu como estudante, gestor do RU ou administrador quero ver o ranking dos pratos mais

						favoritados para entender a demanda pelos pratos do RU
CN06/RF15	6	Registra avaliação do prato	Funcional	Should	Registra avaliação do usuário acerca de algum prato do RU	Eu como estudante quero avaliar um prato do RU para que o gestor do RU saiba quais os pratos mais favoritados pelos estudantes.
CN06/RF16	6	Editar avaliação do prato	Funcional	Should	Permite que o usuário atualize a suas avaliações	Eu como estudante quero reavaliar um prato do RU para que o gestor do RU saiba quais os pratos mais favoritados pelos estudantes de forma atualizada.
RNF1	2-7	Responsividade	Não Funcional	Must	A interface deve ser adaptável para uso em dispositivos móveis (Mobile-first)	Eu como usuário do sistema quero ser capaz de acessar o sistema através do celular ou do computador para facilitar a obtenção das informações disponíveis no sistema.
RNF2	2-7	Persistência de dados	Não Funcional	Must	Os dados devem ser armazenados no SQLite garantindo sua integridade	Eu como estudante, gestor do RU ou administrador quero que os dados sejam salvos para permitir a utilização do sistema.

5 MÉTRICAS E MEDIÇÕES

Esta seção define a abordagem de medição adotada para acompanhar o desenvolvimento e a utilização do sistema AvaliaRU, em conformidade com a proposta do projeto, que busca transformar a publicação passiva do cardápio em um serviço mais interativo, confiável e orientado a dados. O planejamento das medições considera o ciclo de vida do software, a natureza do produto e o contexto de uso no Restaurante Universitário da FGA/UnB.

A estrutura escolhida segue o paradigma GQM (Goal–Question–Metric), no qual cada métrica existe para responder a uma questão ligada a um objetivo de medição previamente definido. Essa abordagem é compatível com o material da disciplina e com as normas de medição e qualidade de software, pois permite explicitar o que será medido, por que será medido, como será calculado, quando será coletado e quais decisões serão tomadas a partir dos resultados.

5.1 GQM de medições

Na leitura deste projeto, as medições foram organizadas em três perspectivas complementares: processo de desenvolvimento, qualidade do produto e qualidade em uso. A primeira acompanha a evolução técnica da equipe; a segunda verifica se as funcionalidades entregues atendem aos requisitos; e a terceira observa se o sistema realmente gera valor para alunos e funcionários do RU.

Tabela 8 – Objetos De Análise

ID	Objeto de análise	Propósito	Foco de qualidade	Ponto de vista	Contexto
OBJ-01	Processo de desenvolvimento	Monitorar e melhorar	Previsibilidade, estabilidade e redução de pendências técnicas	Equipe de desenvolvimento e gestão do projeto	Sprints semanais do projeto AvaliaRU
OBJ-02	Produto de software (AvaliaRU)	Avaliar e garantir	Confiabilidade, correteude funcional e desempenho	Alunos, funcionários do RU e equipe técnica	Ambiente de homologação e uso controlado
OBJ-03	Uso do sistema	Verificar e aprimorar	Usabilidade, engajamento e utilidade prática	Usuários finais do RU	Uso cotidiano no contexto acadêmico

As medições devem ser coletadas ao final de cada Sprint, na revisão de sprint e, quando necessário, durante a execução por meio de logs, consultas ao banco de dados, relatórios de testes e registros de issues. A análise deve considerar a comparação entre o resultado previsto e o resultado realizado, permitindo correções imediatas quando um indicador sair da faixa esperada.

5.1.1 Questões de medição

Q1: O débito técnico do projeto está sob controle ao longo das Sprints?

Q2: As entregas do AvaliaRU apresentam baixo índice de defeitos relevantes?

Q3: Os testes automatizados cobrem adequadamente as funcionalidades críticas?

Q4: O desempenho do sistema é aceitável nas operações mais usadas?

Q5: As notificações personalizadas estão funcionando de forma confiável?

Q6: O sistema está gerando adesão dos usuários às funções de valor (restrições, favoritos e avaliações)?

5.1.2 Métricas, limites e ações

Tabela 9 - Métricas do Software

I D	Métrica	Fórmula / definição	Unidade	Meta esperada	Periodicid ade	Ação quando fora da meta
M 1	Débito técnico acumulado	Quantidade de itens técnicos pendentes classificados como débito técnico ao final da Sprint (refatorações, correções, documentação e ajustes de arquitetura).	Itens/ponto s	Tendência de queda; não aumentar por duas Sprints consecutiva s.	Semanal / ao fim da Sprint	Suspender novas funcionalida des por um ciclo curto para reduzir pendências, refatorar e concluir itens críticos.
M 2	Densidade de defeitos	Número de defeitos encontrados em testes dividido pelo número de funcionalidades concluídas na Sprint.	Defeitos por funcionalid ade	≤ 1 defeito por funcionalid ade; zero defeitos críticos na release.	Ao fim da Sprint Review	Revisar implementaç ão, reforçar pair programmin g e ampliar testes de integração.
M 3	Cobertura de testes automatiza dos	Percentual de linhas ou ramos do código crítico cobertos por testes automatizados.	%	$\geq 80\%$ no conjunto das funcionalid ades críticas.	A cada Pull Request e ao fim da Sprint	Bloquear merge em PRs que reduzam a cobertura e exigir a criação dos testes corresponde ntes.
M 4	Tempo médio de resposta	Média dos tempos de resposta das ações principais: visualizar cardápio, salvar	Segundos (s) / milissegun dos (ms)	Média ≤ 2 s nas operações críticas do ambiente de homologaçã o.	Durante testes e validações	Otimizar consultas, revisar componente s e reduzir chamadas

		avaliação e registrar restrição.				desnecessárias ao banco.
M 5	Taxa de sucesso das notificações	Quantidade de notificações personalizadas exibidas corretamente dividido pela quantidade de notificações geradas.	%	$\geq 98\%$ de sucesso nas notificações essenciais.	Semanal	Revisar regras de filtragem, integração com dados de restrição e tratamento de erros.
M 6	Taxa de adesão do usuário	Percentual de usuários ativos que realizaram ao menos uma ação de valor no período (cadastrar restrição/preferência ou registrar avaliação).	%	Crescimento contínuo; meta inicial definida no piloto, com tendência de aumento ao longo das Sprints.	Mensal	Ajustar usabilidade, chamadas de onboarding e visibilidade das funções principais.

A interpretação dos resultados deve ser feita de forma comparativa, observando a evolução entre Sprints e o impacto real no produto. Indicadores de processo, como débito técnico e cobertura de testes, ajudam a proteger a base do sistema. Indicadores de produto, como defeitos e desempenho, verificam a qualidade externa. Já os indicadores de uso, como notificações e adesão, mostram se o AvaliaRU está cumprindo sua finalidade no contexto do RU.

Quando um indicador ficar fora da meta, a equipe deve registrar o desvio no quadro de tarefas, priorizar a causa raiz e aplicar ações corretivas compatíveis com a gravidade do problema. Esse fluxo mantém as medições integradas ao processo de desenvolvimento e evita que os resultados sirvam apenas como registro, sem geração de melhoria real.

5.2 Considerações de verificação e validação

Conforme discutido nas aulas da disciplina, a qualidade do software deve ser observada tanto por meio de verificação (se o sistema está de acordo com as especificações técnicas) quanto por validação (se atende às necessidades do usuário). Por isso, o plano de medição do AvaliaRU combina métricas internas e externas com testes unitários, de integração, de sistema e de aceitação, além de revisões em pares e análise contínua dos artefatos produzidos.

6 TESTES DE SOFTWARE

Esta seção define a abordagem de qualidade adotada pela equipe para o desenvolvimento da aplicação web AvaliaRU. O objetivo é garantir que as funcionalidades críticas mapeadas no

Backlog — especialmente as avaliações de pratos e os alertas de ingredientes alergênicos — operem de forma confiável e ágil no contexto dos estudantes do FCTE.

6.1 Estratégia de testes contendo:

A estratégia de testes visa identificar e mitigar falhas antes do deploy em produção (AWS), utilizando as seguintes abordagens alinhadas às tecnologias (Next.js, TypeScript, SQLite) e às práticas de Extreme Programming (Integração Contínua) adotadas pelo grupo:

Níveis de testes abordados:

- **Unitário:** Validação isolada de lógicas em TypeScript, como as funções de filtragem de ingredientes alergênicos e o cruzamento de dados para alertas de alimentos favoritos.
- **Integração:** Verificação da comunicação entre o Front-end e o Back-end (Next.js API Routes), garantindo que a avaliação do aluno seja gravada e lida corretamente no banco de dados local (SQLite).
- **Sistema:** Validação do fluxo completo (End-to-End), simulando o estudante acessando a aplicação via navegador para visualizar o cardápio e interagir com as funcionalidades.

Tipos de testes abordados:

- **Funcionais:** Focados em atestar que os requisitos listados no Backlog do Produto funcionam de acordo com o esperado.
- **Não Funcionais (Usabilidade e Desempenho):** Testes de interface para garantir que a aplicação seja responsiva e carregue rapidamente em dispositivos móveis, sem quebra de layout.

Ambientes de testes usados: Os testes serão executados primeiramente no ambiente local dos desenvolvedores. O avanço para a integração segue estritamente a nossa **política de branches e commits**: o desenvolvimento ocorre em *feature branches* e, ao abrir um *Pull Request* para a *branch* principal (main), uma esteira de Integração Contínua (CI) é disparada. O código só é mesclado se os testes passarem no ambiente de CI, garantindo a estabilidade antes da homologação e do deploy final na AWS.

Formas de análise e Resultados (Previsto x Realizado): A análise se dará pela execução do Roteiro de Testes e a comparação direta entre os Resultados Previstos (critérios de aceite) e os Resultados Realizados. O raciocínio aplicado é: se Previsto for diferente de Realizado ($\text{Previsto} \neq \text{Realizado}$), o teste não passou e um defeito foi identificado. Esse defeito será inserido no Kanban para reparo imediato pela equipe, e o teste será repetido em ciclos até que $\text{Previsto} = \text{Realizado}$, indicando a aprovação do teste e a ausência do defeito.

6.2 Roteiro de teste:

- A tabela abaixo constitui o planejamento dos casos de testes. As evidências objetivas (prints e logs) serão anexadas na coluna de Registro durante a execução das Sprints.

Tabela 10 – Roteiro de Casos de Testes do AvaliaRU

ID do Teste	Nome e Objetivo do Teste	Nível / Tipo	Precondições	Definição de Aceito/Rejeitado	Registro (Previsto x Realizado)	Reparos Executados
CT01	Exibição do Cardápio: Exibir o cardápio diário do RU FCTE.	Sistema / Funcional	Banco de dados atualizado com o cardápio.	Aceito se: A página renderizar os pratos e guarnições correspondentes à data atual.	[A preencher na Sprint 3]	[A preencher]
CT02	Alerta de Alergênicos: Notificar o usuário sobre ingredientes alergênicos.	Unidade / Funcional	Usuário possuir restrição cadastrada (ex: Lactose).	Aceito se: A interface aplicar um alerta visual sobre pratos que contenham os alergênicos.	[A preencher na Sprint 4]	[A preencher]
CT03	Cadastro de Favoritos: Registrar os alimentos favoritos do usuário.	Integração / Funcional	Sistema online.	Aceito se: O usuário conseguir adicionar um prato à sua lista de favoritos no banco de dados.	[A preencher na Sprint 4]	[A preencher]
CT04	Notificação de Favorito: Notificar o usuário quando o prato favorito for servido.	Sistema / Funcional	O prato favorito estar no cardápio do dia.	Aceito se: O sistema exibir corretamente um alerta ou destaque visual.	[A preencher na Sprint 5]	[A preencher]
CT05	Registro de Avaliação: Registrar avaliação do usuário acerca de um prato.	Integração / Funcional	Prato do dia disponível na interface.	Aceito se: A nota e o comentário forem salvos no SQLite e o sistema exibir mensagem de sucesso.	[A preencher na Sprint 6]	[A preencher]
CT06	Usabilidade Mobile: Testar a responsividade da aplicação web.	Sistema / Não Funcional	Acessar via navegador de celular (ex: Chrome Mobile).	Aceito se: Elementos de interface (botões de avaliação) se adaptarem à tela sem quebra de layout.	[A preencher na Sprint 7]	[A preencher]

7 REFERÊNCIAS BIBLIOGRÁFICAS

1. BECK, K. **Programação extrema (XP) explicada**. Porto Alegre: Bookman, 2004.

2. SOMMERVILLE, I. **Engenharia de software**. 10. Ed. São Paulo: Pearson Addison Wesley, 2019.
3. SCHWABER, K; SUTHERLAND, J. **O guia do Scrum: o guia definitivo para o Scrum: as regras do jogo**. [S.l.: s.n.], 2020.
4. GABRIEL, José; NICOLAS, Rafael. **Pesquisa de opinião dos alunos**. Brasília, 2026. 1 questionário aplicado em maio de 2026.